

explicit Keyword

explicit keyword — why use it?

The `explicit` keyword is mainly used with **constructors that can be called with a single argument**. Its purpose is to **prevent unwanted implicit conversions**.

1. The Problem: Implicit Conversion

Suppose we have:

```
class Student {  
public:  
    Student(int age) {  
        // ...  
    }  
};
```

Now:

```
Student s(20);    // Normal construction
```

But C++ also allows:

```
Student s = 20;
```

What happened here?

C++ implicitly converts:

```
int 20  
↓  
Student(20)  
↓  
Student object
```

This is called an **implicit conversion**.

2. explicit Prevents This

Write:

```
class Student {  
public:  
    explicit Student(int age) {  
        // ...  
    }  
};
```

Now:

```
Student s(20);    // ✓  
Student s{20};    // ✓  
Student s = 20;   // ✗
```

The constructor can still be used normally, but C++ cannot silently use it for implicit conversions.

Mental model

`explicit` = "Don't automatically convert other types into my class."

3. Why Is Implicit Conversion Dangerous?

Imagine:

```
class Distance {  
public:  
    Distance(int meters) {}  
};
```

And:

```
void printDistance(Distance d) {  
}
```

Now you accidentally write:

```
printDistance(100);
```

C++ says:

```
100  
  ↓ implicit conversion  
Distance(100)  
  ↓  
printDistance()
```

Maybe that's what you intended.

But maybe you accidentally passed an integer where a `Distance` object was expected.

With:

```
class Distance {  
public:  
    explicit Distance(int meters) {}  
};
```

this becomes:

```
printDistance(100); // ✗
```

You must explicitly construct it:

```
printDistance(Distance(100)); // ☑
```

This makes the code's intent clearer.

4. `explicit` Doesn't Prevent Normal Construction

This is important.

```
class Student {  
public:  
    explicit Student(int age) {}  
};
```

You can still do:

```
Student s1(20);    // ☑  
Student s2{20};    // ☑
```

You're only preventing **implicit conversion**:

```
Student s3 = 20;    // ✗
```

So:

```
explicit  
  ↓  
Does NOT mean "constructor can't be called"  
  ↓  
Means "constructor can't be used for implicit conversion"
```

5. Very Common Interview Example

Without `explicit`:

```
class Number {  
public:  
    Number(int x) {}  
};  
  
void foo(Number n) {}  
  
foo(10);    // ☑
```

C++ automatically does:

```
foo(Number(10));
```

With `explicit`:

```
class Number {
public:
    explicit Number(int x) {}
};

foo(10);    // ✗
```

You need:

```
foo(Number(10)); // ✓
```

6. What About Two-Argument Constructors?

Traditionally, `explicit` is particularly relevant to constructors that can be invoked with **one argument**.

For example:

```
class A {
public:
    A(int x, int y = 10) {}
};
```

This can effectively be called with one argument:

```
A a = 5;
```

So `explicit` is useful here too:

```
class A {
public:
    explicit A(int x, int y = 10) {}
};
```

Now:

```
A a = 5;    // ✗
A a(5);     // ✓
```

7. `explicit` Conversion Operators

`explicit` isn't limited to constructors.

It can also be used with conversion operators:

```
class Student {
public:
    explicit operator int() const {
        return 20;
    }
};
```

Then:

```
Student s;

int x = s;           // ✗ implicit conversion
int y = static_cast<int>(s); // ✓ explicit conversion
```

This is less commonly asked than constructors, but good to know.

8. Interview Question

Q: Why do we use `explicit` constructors?

Good interview answer:

"`explicit` prevents a constructor from being used for implicit type conversions. It is useful for avoiding unintended conversions and making the programmer's intent explicit."

Example:

```
class Distance {  
public:  
    explicit Distance(int meters) {}  
};
```

Therefore:

```
Distance d(100); // ☒  
Distance d = 100; // ☒
```

★ Quick Cheat Sheet

CodeMeaning `Student s(10);` Direct initialization `Student s{10};` List initialization `Student s = 10;` Copy initialization → implicit conversion possible `explicit Student(int);` Prevents implicit conversion `Student s(10);` with `explicit` ☒ `Student s = 10;` with `explicit` ☒

Remember this one line:

`explicit` doesn't prevent construction — it prevents implicit conversion through the constructor.